

Reinforcement Learning for Hide-and-Seek using Curriculum Learning in Godot

Cathal Hardy | BEng (Hons) Software & Electronic Engineering | Supervisor: Brian O'Shea | Atlantic Technological University

Overview

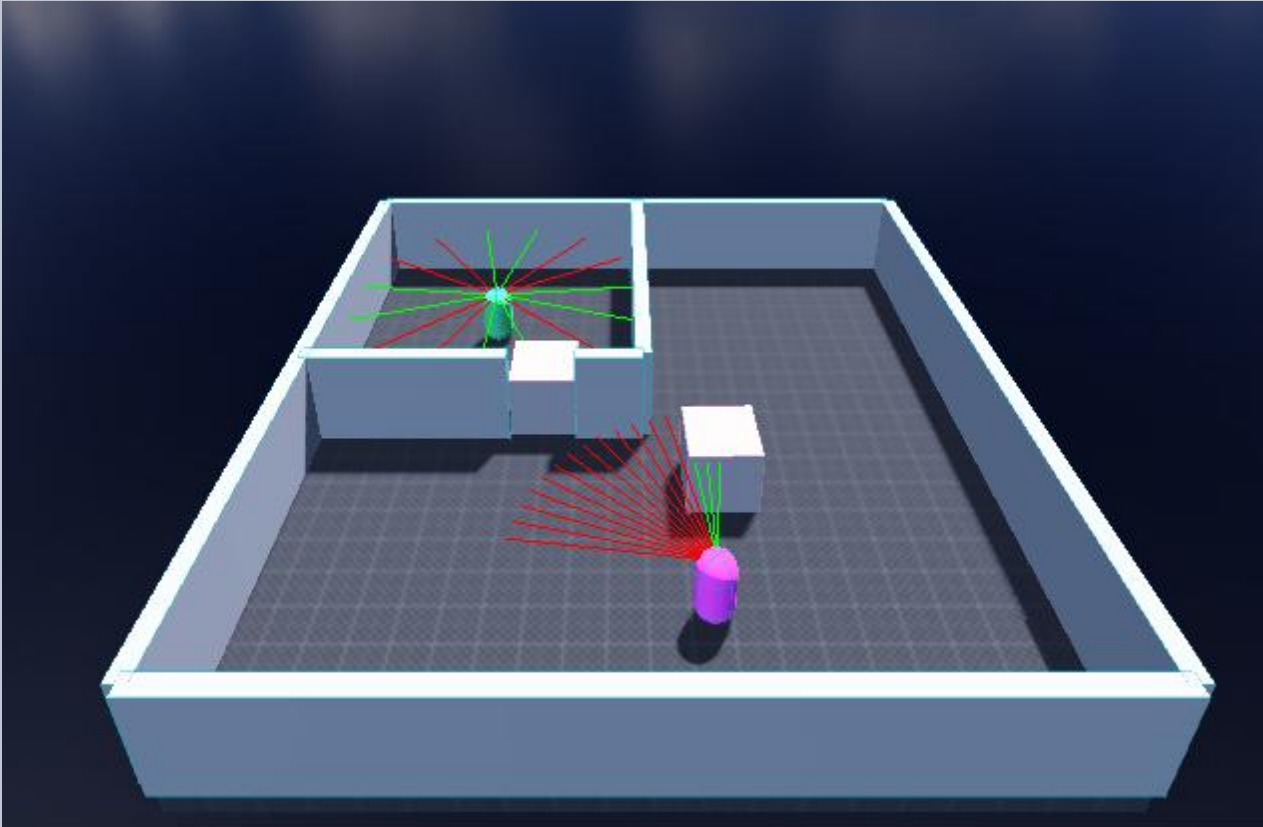
This project builds a hide-and-seek simulation in Godot where two AI agents learn through deep reinforcement learning. A seeker must catch the hider using only sensor observations, while the hider must survive as long as possible.

The seeker is trained progressively through increasing levels of difficulty, starting with fixed spawn points, then randomised positions, then a walled arena with walls and moveable obstacles. Once sufficiently competent, the hider is then trained against the seeker's learned policy, developing evasion strategies through competition.

Both agents use Proximal Policy Optimization (PPO) with no hand-coded rules, developing emergent strategies through experience alone.

Training Pipeline

- Stage 0** Fixed spawn points. Seeker learns basic movement and catching.
- Stage 1** Randomised spawn points. Policy generalises across start positions.
- Stage 2** Added inner walls and moveable box obstacles. Tests occlusion-aware navigation.
- Hider** Trained against the pre-trained seeker's learned policy. A 40-step preparation phase gives the hider a head start before the seeker acts.

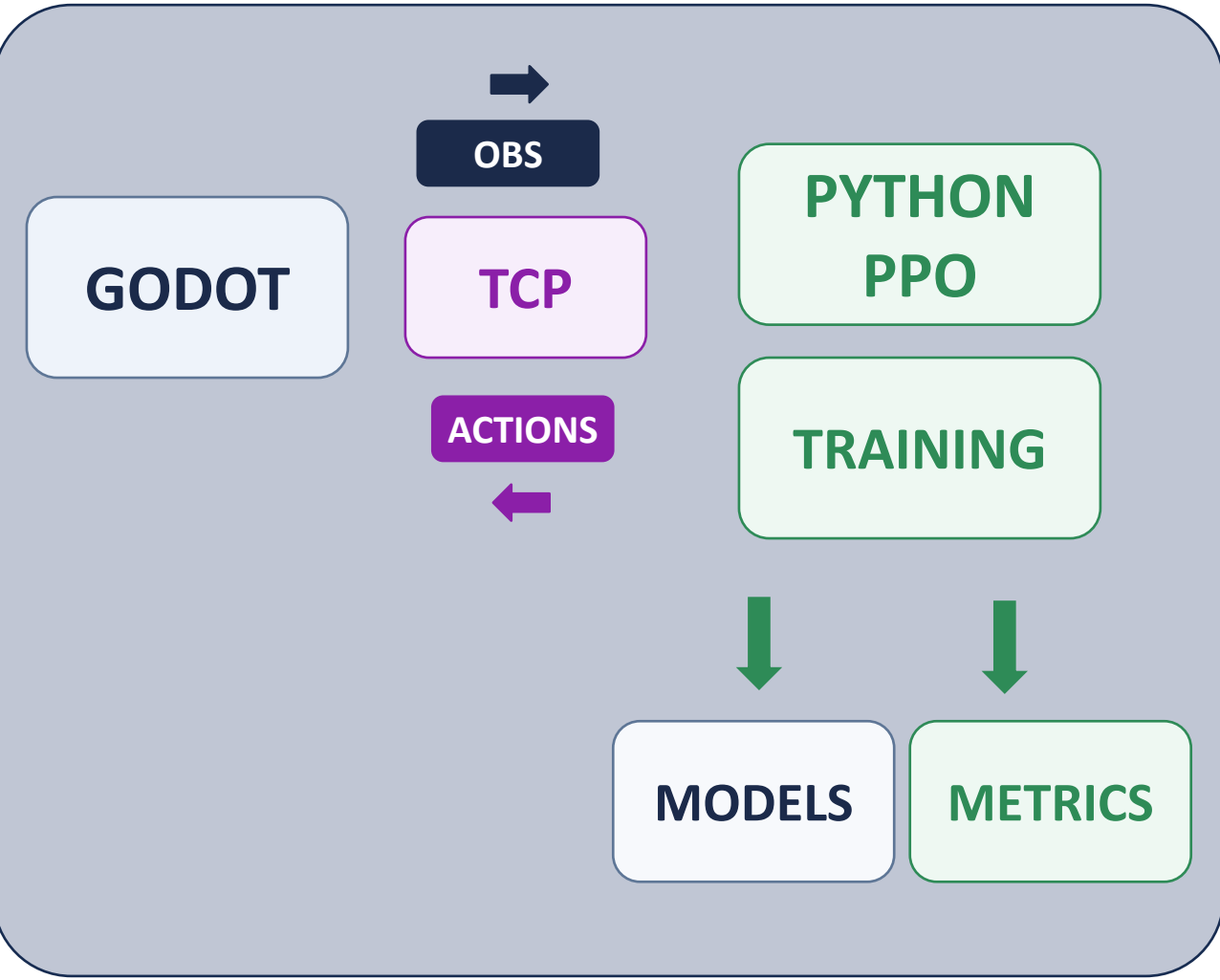


Observation Space (57 features)

Velocity (x, z)	2
Heading (sin θ , cos θ)	2
Carried-box flag	1
Agent position (x, z)	2
Environment raycasts (360°)	16
FOV raycasts (135°)	23
Sees-target flag	1
Target motion features	4
Target position (bearing, dist, abs x/z)	5
Episode progress	1
Total	57

System Architecture

Godot and Python communicate via a TCP socket bridge, exchanging observations, actions, and rewards as JSON messages each step.

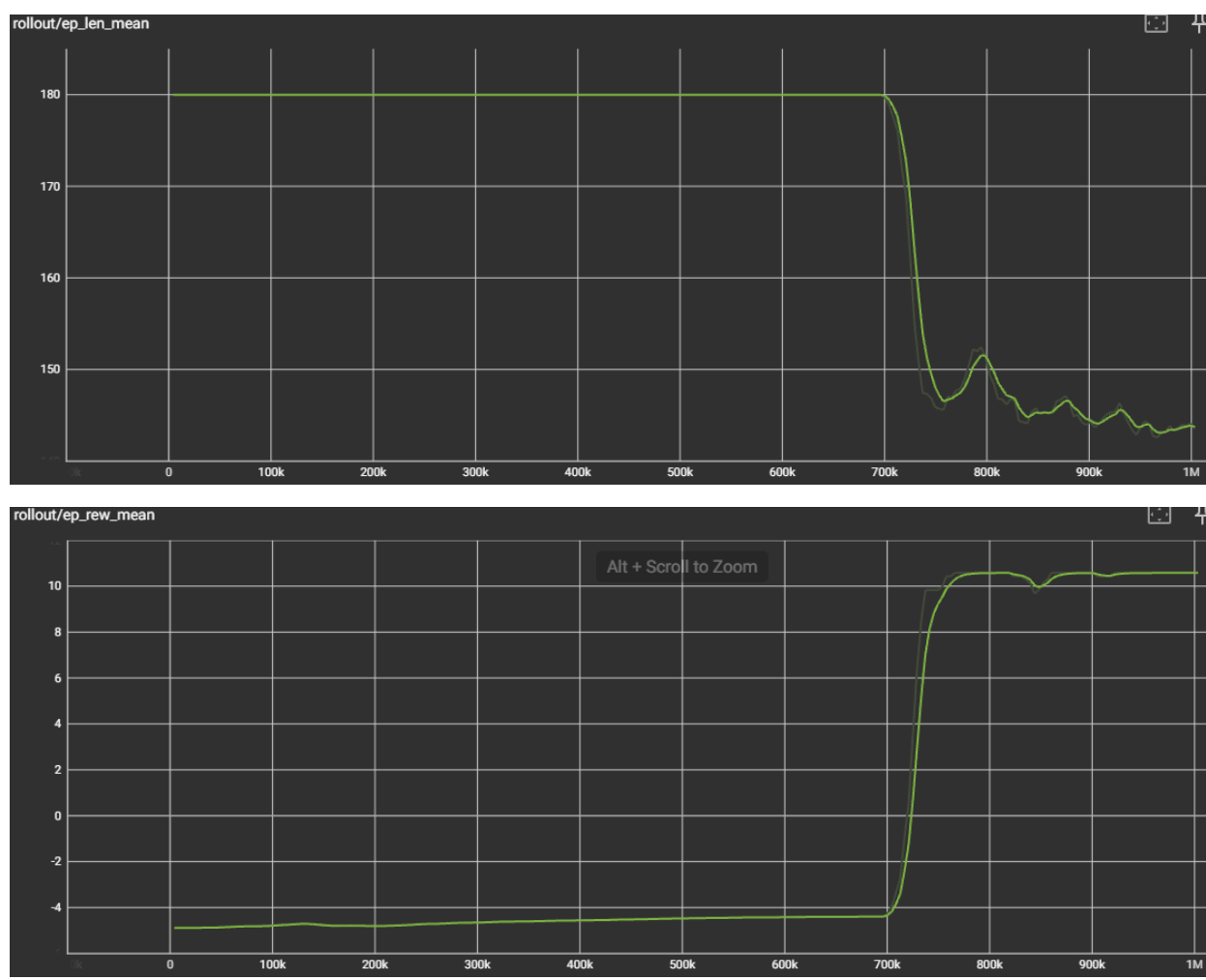


- Godot sends observations/rewards to Python over TCP; the Python agent returns an action, and the environment steps forward.
- Two parallel environments run simultaneously during seeker training to collect data faster.
- The PPO algorithm updates the policy using rollout batches of 2048 steps.
- TensorBoard tracks episode reward and episode length in real time, making it easy to spot when training stalls or converges.

Technologies & Tools

- Godot**
3D game engine for the hide-and-seek arena. Provides physics, raycasts, CharacterBody3D agents, and a TCP server node.
- GDScript**
Godot scripting language. Used for agent logic, reward calculation, environment configuration, and TCP bridge protocol.
- Python**
Hosts the GodotEnv wrapper, PPO training scripts, and model loading for inference during evaluation.
- Stable-Baselines3**
Provides the PPO implementation, SubprocVecEnv, Monitor wrapper, and checkpoint callbacks used throughout training.
- TensorBoard**
Live view of training metrics to track progress, spot when learning stalls, and compare curriculum stages.
- NumPy**
Observation and action array handling within the Python environment wrapper and training pipeline.

Results - Seeker



- ep_len_mean stayed near 180 until about 700k steps, meaning most episodes lasted the full length.
- It then dropped to about 143 by 1M steps, showing the seeker was catching the hider faster.
- ep_rew_mean stayed negative until about 700k steps, then rose to around 10, showing clear improvement.

Results - Hider



- The hider showed steady improvement across the full 1M training steps.
- Both ep_rew_mean and ep_len_mean increased consistently from start to finish.
- This suggests the hider got better at staying hidden from the seeker and surviving longer.

Video Demo



Scan to watch the project demo playlist